modeling-core-mahin

Mahin Roddur

2026-04-08

```r
library(here)
```

```
## Warning: package 'here' was built under R version 4.5.3
```

```
## here() starts at C:/Users/roddu/Downloads/glassdoor-company-reviews
```

```r
data <- read.csv(here("data", "processed", "reviews_with_sent_scores.csv"))
```

```r
data <- data %>%
  mutate(
    employee_length_bin = case_when(
      employee_length <= 1 ~ "short",
      employee_length < 4 ~ "medium",
      TRUE ~ "long"
    ),
    employee_length_bin = factor(
      employee_length_bin,
      levels = c("short", "medium", "long"),
      ordered = TRUE
    ))
```

```r
ggplot(data, aes(x = employee_length_bin)) +
  geom_bar() +
  theme_minimal() +
  labs(title = "Distribution of Employee Tenure Categories")
```

The bins are not heavily imbalanced. Yet, it was hard to predict, as we will see.

```
features_base <- c(
  # Ratings
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion",

  # Context
  "employee_job_quit",
  "covid_related_review",
  "rating_year",

  # Missingness
  "career_missing",

  # Engagement
  "helpfulness_score",

  # Text structure
  "summary_wc",
  "pros_wc",
  "cons_wc",
  "advice_wc"
```

```
)
```

Scaling helper

```
scale_train_test <- function(train_df, test_df, feature_cols) {
  train_scaled <- train_df
  test_scaled  <- test_df

  train_means <- sapply(train_df[feature_cols], mean, na.rm = TRUE)
  train_sds   <- sapply(train_df[feature_cols], sd, na.rm = TRUE)

  train_sds[train_sds == 0 | is.na(train_sds)] <- 1

  train_scaled[feature_cols] <- scale(train_df[feature_cols],
                                      center = train_means,
                                      scale = train_sds)

  test_scaled[feature_cols] <- scale(test_df[feature_cols],
                                     center = train_means,
                                     scale = train_sds)

  list(
    train = train_scaled,
    test = test_scaled,
    center = train_means,
    scale = train_sds
  )
}
```

Train

```
split_data <- function(df, target, p = 0.8, seed = 123) {
  set.seed(seed)
  idx <- createDataPartition(df[[target]], p = p, list = FALSE)
  list(
    train = df[idx, ],
    test  = df[-idx, ]
  )
}

rmse_vec <- function(actual, predicted) {
  sqrt(mean((actual - predicted)^2, na.rm = TRUE))
}
```

```
run_tenure_lm <- function(df, feature_cols, target = "employee_length", seed = 123) {
  df_model <- data %>%
  dplyr::select(employee_length, all_of(features_base))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
```

```r
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  formula_lm <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- lm(formula_lm, data = train_scaled)

  preds <- predict(model, newdata = test_scaled)
  rmse  <- rmse_vec(test_scaled[[target]], preds)

  coef_tbl <- broom::tidy(model) %>%
    filter(term != "(Intercept)") %>%
    arrange(desc(abs(estimate)))

  list(
    model = model,
    rmse = rmse,
    predictions = preds,
    actual = test_scaled[[target]],
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

tenure_lm_results <- run_tenure_lm(
  df = data,
  feature_cols = features_base,
  target = "employee_length"
)

tenure_lm_results$rmse
```

```
## [1] 4.455652
```

```r
tenure_lm_results$coefficients
```

```
## # A tibble: 15 x 5
##    term                          estimate std.error statistic  p.value
##    <chr>                            <dbl>     <dbl>     <dbl>    <dbl>
##  1 rating_senior_leadership     -0.919       0.262   -3.51   0.000481
##  2 rating_culture_values         0.504       0.267    1.89   0.0597
##  3 rating_year                  -0.472       0.156   -3.03   0.00251
##  4 career_missing               -0.423       0.193   -2.19   0.0289
##  5 rating_compensation_benefits  0.292       0.196    1.49   0.137
##  6 cons_wc                      -0.279       0.167   -1.67   0.0952
##  7 rating_career_opportunities   0.258       0.170    1.52   0.129
##  8 helpfulness_score            -0.194       0.162   -1.20   0.231
```

```
##  9 covid_related_review          0.150       0.152    0.989  0.323
## 10 employee_job_quit             0.141       0.155    0.907  0.365
## 11 pros_wc                       0.127       0.160    0.794  0.427
## 12 advice_wc                     0.111       0.182    0.608  0.543
## 13 summary_wc                    0.0412      0.152    0.271  0.786
## 14 rating_work_life             -0.0271      0.201   -0.135  0.893
## 15 rating_diversity_inclusion    0.00513     0.210    0.0244 0.981
```

```r
summary(tenure_lm_results$model)
```

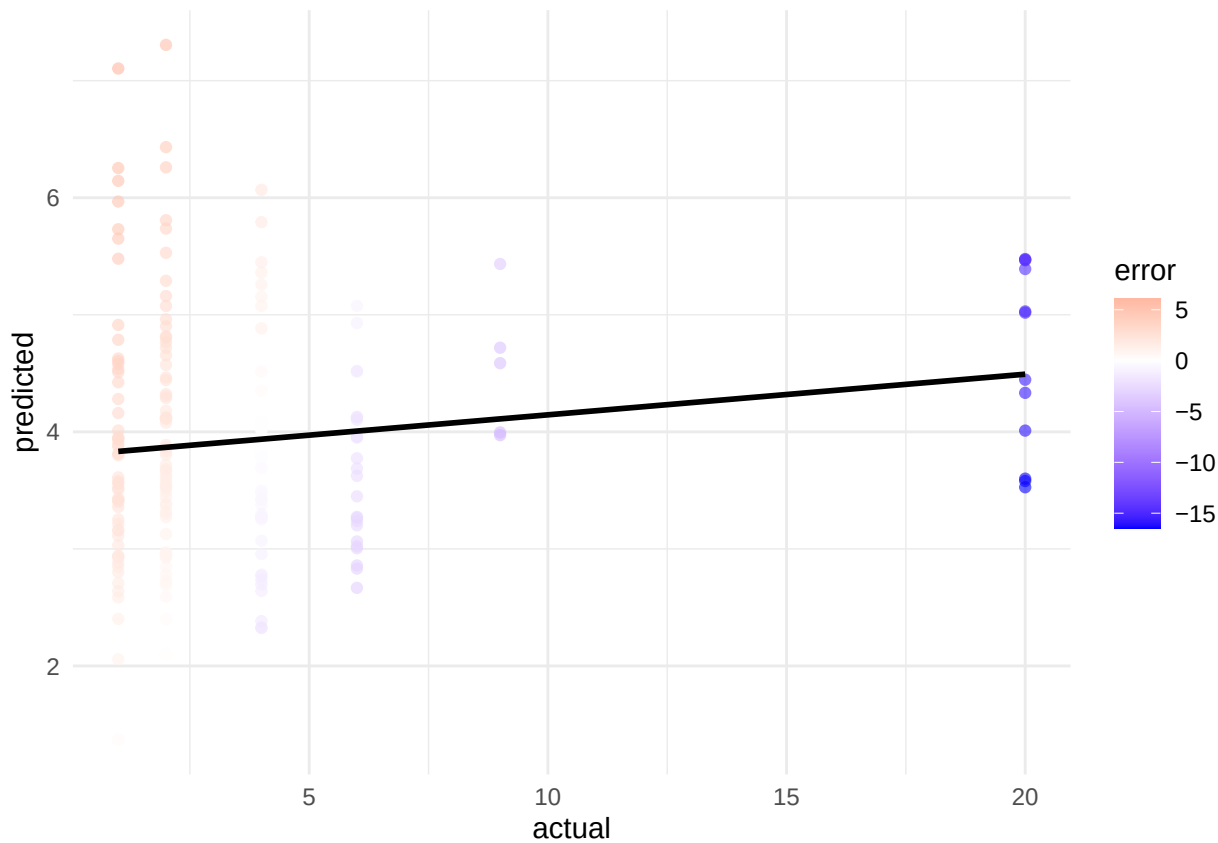
```
##
## Call:
## lm(formula = formula_lm, data = train_scaled)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -5.0599 -2.2492 -1.1693  0.7837 17.7216
##
## Coefficients:
##                              Estimate Std. Error t value Pr(>|t|)
## (Intercept)                  3.834165   0.143648  26.691  < 2e-16 ***
## rating_culture_values        0.503975   0.267252   1.886 0.059694 .
## rating_work_life            -0.027141   0.200966  -0.135 0.892605
## rating_career_opportunities  0.258492   0.170194   1.519 0.129212
## rating_compensation_benefits 0.291688   0.195716   1.490 0.136529
## rating_senior_leadership    -0.919206   0.262181  -3.506 0.000481 ***
## rating_diversity_inclusion   0.005129   0.210404   0.024 0.980558
## employee_job_quit            0.140943   0.155355   0.907 0.364562
## covid_related_review         0.150381   0.152113   0.989 0.323157
## rating_year                 -0.471937   0.155686  -3.031 0.002515 **
## career_missing              -0.422518   0.193072  -2.188 0.028934 *
## helpfulness_score           -0.193745   0.161560  -1.199 0.230807
## summary_wc                   0.041250   0.152103   0.271 0.786310
## pros_wc                      0.126718   0.159610   0.794 0.427479
## cons_wc                     -0.278869   0.166915  -1.671 0.095175 .
## advice_wc                    0.110776   0.182058   0.608 0.543055
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 4.068 on 786 degrees of freedom
## Multiple R-squared:  0.05694,    Adjusted R-squared:  0.03894
## F-statistic: 3.164 on 15 and 786 DF,  p-value: 4.498e-05
```

Abysmal R-squared 0.03894. RMSE is low but the model is barely better than randomly guessing.

```r
df_plot <- data.frame(
  actual = tenure_lm_results$actual,
  predicted = tenure_lm_results$predictions
)

df_plot$error <- df_plot$predicted - df_plot$actual

ggplot(df_plot, aes(x = actual, y = predicted, color = error)) +
```

```r
  geom_point(alpha = 0.6) +
  geom_smooth(method = "lm", se = FALSE, color = "black") +
  scale_color_gradient2(low = "blue", mid = "white", high = "red") +
  theme_minimal()
```

```
## `geom_smooth()` using formula = 'y ~ x'
```



Looks goofy, making employee_length into ordinal bins might help. The model systematically underestimates long-tenure employees and overestimates short-tenure employees.

Really low adjusted R-squared: 0.03894. Attempting LASSO but doubt it will help.

```r
library(glmnet)

run_lasso_tenure <- function(df, feature_cols, target = "employee_length", seed = 123) {
  df_model <- data %>%
  dplyr::select(employee_length, all_of(features_base))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test
```

```r
  x_train <- model.matrix(
    as.formula(paste(target, "~", paste(feature_cols, collapse = " + "))),
    data = train_scaled
  )[ , -1, drop = FALSE]

  y_train <- train_scaled[[target]]

  x_test <- model.matrix(
    as.formula(paste(target, "~", paste(feature_cols, collapse = " + "))),
    data = test_scaled
  )[ , -1, drop = FALSE]

  y_test <- test_scaled[[target]]

  cv_model <- cv.glmnet(
    x = x_train,
    y = y_train,
    alpha = 1,
    family = "gaussian"
  )

  best_lambda <- cv_model$lambda.min

  preds <- as.numeric(predict(cv_model, s = best_lambda, newx = x_test))
  rmse  <- rmse_vec(y_test, preds)

  coef_tbl <- as.matrix(coef(cv_model, s = "lambda.min")) %>%
    as.data.frame()

  colnames(coef_tbl) <- "estimate"
  coef_tbl$term <- rownames(coef_tbl)

  coef_tbl <- coef_tbl %>%
    dplyr::filter(term != "(Intercept)", estimate != 0) %>%
    dplyr::arrange(desc(abs(estimate)))

  list(
    model = cv_model,
    lambda = best_lambda,
    rmse = rmse,
    predictions = preds,
    actual = y_test,
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

tenure_lasso_results <- run_lasso_tenure(
  df = data,
  feature_cols = features_base,
  target = "employee_length"
)
```

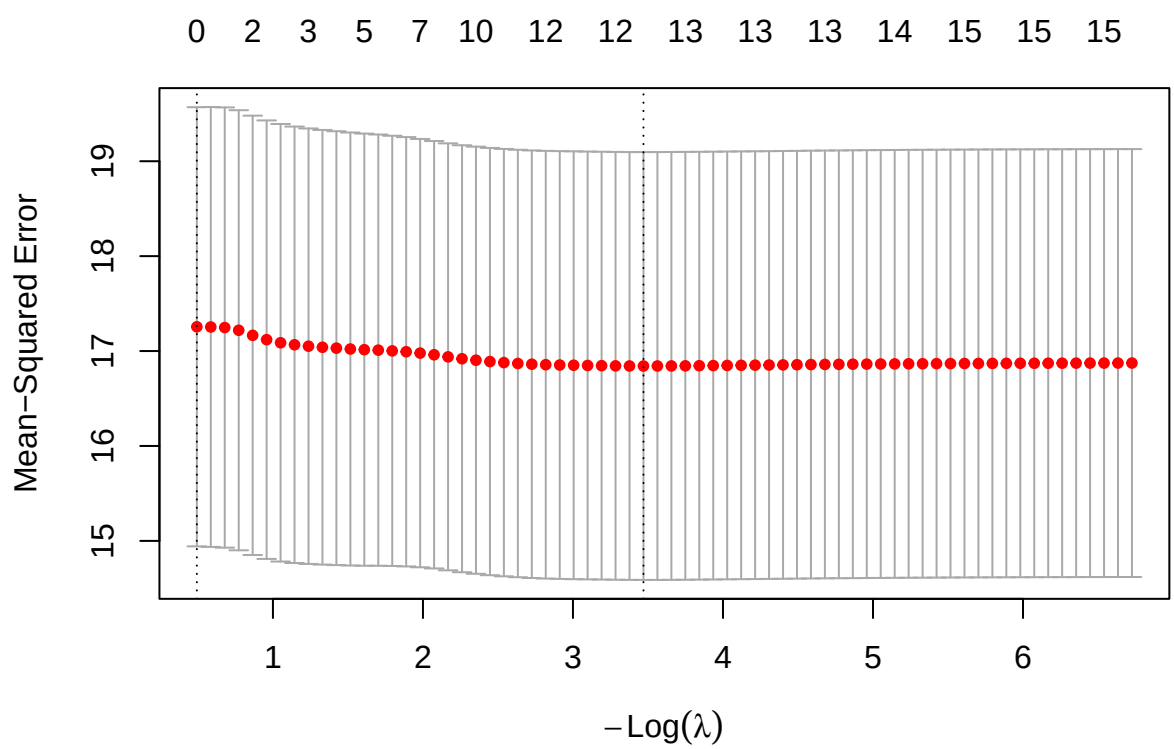
```r
tenure_lasso_results$rmse
```

```
## [1] 4.436089
```

```r
tenure_lasso_results$coefficients
```

```
##                                  estimate                         term
## rating_senior_leadership     -0.719392174     rating_senior_leadership
## rating_year                  -0.464149010                  rating_year
## rating_culture_values         0.384995138        rating_culture_values
## career_missing               -0.364807906               career_missing
## rating_compensation_benefits  0.256696021 rating_compensation_benefits
## cons_wc                      -0.229752468                      cons_wc
## rating_career_opportunities   0.187549805  rating_career_opportunities
## helpfulness_score            -0.158453367            helpfulness_score
## covid_related_review          0.109878818         covid_related_review
## advice_wc                     0.092720580                    advice_wc
## employee_job_quit             0.085246360            employee_job_quit
## pros_wc                       0.079536871                      pros_wc
## summary_wc                    0.003253816                   summary_wc
```

```r
plot(tenure_lasso_results$model)
```



Regularization is cooking without the stove on. MSE barely changes across lambda. Give it up.

Ordinal Regression (polr) for employee_length

```
run_tenure_ordinal <- function(df, feature_cols, target = "length_bin", seed = 123) {
  df_model <- df %>% dplyr::select(all_of(c(target, feature_cols)))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  formula_ord <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- polr(formula_ord, data = train_scaled, Hess = TRUE)

  prob_preds <- predict(model, newdata = test_scaled, type = "probs")
  class_preds <- predict(model, newdata = test_scaled, type = "class")

  acc <- mean(as.character(class_preds) == as.character(test_scaled[[target]]), na.rm = TRUE)

  coef_tbl <- broom::tidy(model) %>%
    filter(!grepl("\\|", term)) %>%
    arrange(desc(abs(estimate)))

  list(
    model = model,
    accuracy = acc,
    class_predictions = class_preds,
    actual = test_scaled[[target]],
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

tenure_ord_results <- run_tenure_ordinal(
  df = data,
  feature_cols = features_base,
  target = "employee_length_bin"
)

tenure_ord_results$accuracy
```

```
## [1] 0.4070352
```

```
tenure_ord_results$coefficients
```

```
## # A tibble: 15 x 5
```

```
##    term                          estimate std.error statistic coef.type
##    <chr>                            <dbl>     <dbl>     <dbl> <chr>
##  1 rating_senior_leadership      -0.358     0.121     -2.96  coefficient
##  2 rating_culture_values          0.228     0.123      1.85  coefficient
##  3 rating_compensation_benefits   0.125     0.0908     1.38  coefficient
##  4 career_missing                -0.122     0.0912    -1.34  coefficient
##  5 rating_diversity_inclusion     0.122     0.0952     1.28  coefficient
##  6 rating_year                   -0.113     0.0739    -1.53  coefficient
##  7 employee_job_quit             -0.0826    0.0703    -1.17  coefficient
##  8 pros_wc                        0.0704    0.0779     0.904 coefficient
##  9 rating_career_opportunities   -0.0676    0.0798    -0.847 coefficient
## 10 cons_wc                       -0.0626    0.0803    -0.779 coefficient
## 11 summary_wc                    -0.0421    0.0692    -0.609 coefficient
## 12 advice_wc                     -0.0379    0.0872    -0.435 coefficient
## 13 rating_work_life              -0.0361    0.0915    -0.395 coefficient
## 14 helpfulness_score              0.0301    0.0738     0.408 coefficient
## 15 covid_related_review          -0.0107    0.0692    -0.155 coefficient
```

```r
summary(tenure_ord_results$model)
```

```
## Call:
## polr(formula = formula_ord, data = train_scaled, Hess = TRUE)
##
## Coefficients:
##                                 Value Std. Error t value
## rating_culture_values         0.22758    0.12315  1.8479
## rating_work_life             -0.03610    0.09146 -0.3947
## rating_career_opportunities  -0.06761    0.07982 -0.8470
## rating_compensation_benefits  0.12509    0.09080  1.3777
## rating_senior_leadership     -0.35782    0.12087 -2.9605
## rating_diversity_inclusion    0.12181    0.09518  1.2798
## employee_job_quit            -0.08255    0.07031 -1.1740
## covid_related_review         -0.01072    0.06922 -0.1548
## rating_year                  -0.11310    0.07393 -1.5299
## career_missing               -0.12214    0.09119 -1.3394
## helpfulness_score             0.03014    0.07385  0.4082
## summary_wc                   -0.04214    0.06922 -0.6089
## pros_wc                       0.07043    0.07795  0.9036
## cons_wc                      -0.06258    0.08031 -0.7793
## advice_wc                    -0.03793    0.08718 -0.4351
##
## Intercepts:
##              Value    Std. Error t value
## short|medium  -1.1132   0.0825   -13.4971
## medium|long    0.3091   0.0724     4.2691
##
## Residual Deviance: 1697.466
## AIC: 1731.466
```

40% accuracy still pretty bad, better than random (0.33). Huge variance in the bins (1 vs 20 years). Employee tenure might be just hard to predict with the variables available in the dataset.

Does NLP on the text fields help?

Logistic Regression on flag_recommend_frend

```
run_recommend_logit <- function(df, feature_cols, target = "flags_recommend_frend", seed = 123) {
  df_model <- df %>%
    dplyr::select(all_of(c(target, feature_cols)))

  df_model[[target]] <- factor(df_model[[target]], levels = c("NEGATIVE", "POSITIVE"))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test

  scaled <- scale_train_test(train_df, test_df, feature_cols)
  train_scaled <- scaled$train
  test_scaled  <- scaled$test

  train_scaled[[target]] <- factor(train_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))
  test_scaled[[target]]  <- factor(test_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))

  formula_logit <- as.formula(
    paste(target, "~", paste(feature_cols, collapse = " + "))
  )

  model <- glm(formula_logit, data = train_scaled, family = binomial)

  prob_preds <- predict(model, newdata = test_scaled, type = "response")

  class_preds <- ifelse(prob_preds >= 0.5, "POSITIVE", "NEGATIVE")
  class_preds <- factor(class_preds, levels = levels(test_scaled[[target]]))

  acc <- mean(class_preds == test_scaled[[target]], na.rm = TRUE)

  roc_obj <- pROC::roc(
    response = test_scaled[[target]],
    predictor = prob_preds,
    levels = c("NEGATIVE", "POSITIVE"),
    direction = "<"
  )
  auc_val <- as.numeric(pROC::auc(roc_obj))

  coef_tbl <- broom::tidy(model) %>%
    dplyr::filter(term != "(Intercept)") %>%
    dplyr::mutate(odds_ratio = exp(estimate)) %>%
    dplyr::arrange(desc(abs(estimate)))

  list(
    model = model,
    auc = auc_val,
    accuracy = acc,
    roc = roc_obj,
    probabilities = prob_preds,
    predicted_class = class_preds,
    actual = test_scaled[[target]],
```

```r
    coefficients = coef_tbl,
    train = train_scaled,
    test = test_scaled
  )
}

recommend_results <- run_recommend_logit(
  df = data,
  feature_cols = features_base,
  target = "flags_recommend_frend"
)

recommend_results$auc
```

```
## [1] 0.9117368
```

```r
recommend_results$accuracy
```

```
## [1] 0.8643216
```

```r
recommend_results$coefficients
```

```
## # A tibble: 15 x 6
##    term                          estimate std.error statistic  p.value odds_ratio
##    <chr>                            <dbl>     <dbl>     <dbl>    <dbl>      <dbl>
##  1 helpfulness_score            -1.34e+0     0.403 -3.33     8.82 e-4      0.262
##  2 rating_culture_values         9.34e-1     0.223  4.19     2.80 e-5      2.54
##  3 rating_senior_leadership      8.36e-1     0.200  4.19     2.78 e-5      2.31
##  4 cons_wc                      -6.32e-1     0.290 -2.18     2.91 e-2      0.532
##  5 rating_career_opportunities   5.96e-1     0.176  3.39     6.97 e-4      1.82
##  6 pros_wc                       5.40e-1     0.158  3.43     6.12 e-4      1.72
##  7 rating_work_life              5.28e-1     0.174  3.04     2.40 e-3      1.70
##  8 career_missing               -2.99e-1     0.178 -1.68     9.22 e-2      0.742
##  9 employee_job_quit            -2.93e-1     0.142 -2.07     3.86 e-2      0.746
## 10 rating_compensation_benefits  2.86e-1     0.184  1.56     1.20 e-1      1.33
## 11 advice_wc                     6.79e-2     0.185  0.368    7.13 e-1      1.07
## 12 covid_related_review         -5.00e-2     0.151 -0.332    7.40 e-1      0.951
## 13 rating_year                  -2.57e-2     0.133 -0.194    8.46 e-1      0.975
## 14 summary_wc                   -1.19e-2     0.158 -0.0752   9.40 e-1      0.988
## 15 rating_diversity_inclusion   -6.19e-5     0.199 -0.000312 1.000e+0      1.000
```

```r
summary(recommend_results$model)
```

```
##
## Call:
## glm(formula = formula_logit, family = binomial, data = train_scaled)
##
## Coefficients:
##                                Estimate Std. Error z value Pr(>|z|)
## (Intercept)                  -2.0353315  0.2208382  -9.216  < 2e-16 ***
## rating_culture_values         0.9335599  0.2228546   4.189 2.80e-05 ***
```

```
## rating_work_life              0.5280146  0.1739360   3.036 0.002400 **
## rating_career_opportunities   0.5961229  0.1758109   3.391 0.000697 ***
## rating_compensation_benefits  0.2858576  0.1837258   1.556 0.119734
## rating_senior_leadership      0.8361815  0.1995447   4.190 2.78e-05 ***
## rating_diversity_inclusion   -0.0000619  0.1987171   0.000 0.999751
## employee_job_quit            -0.2932859  0.1417906  -2.068 0.038598 *
## covid_related_review         -0.0499544  0.1506081  -0.332 0.740127
## rating_year                  -0.0257478  0.1328943  -0.194 0.846375
## career_missing               -0.2990364  0.1776046  -1.684 0.092236 .
## helpfulness_score            -1.3405740  0.4030962  -3.326 0.000882 ***
## summary_wc                   -0.0118824  0.1580748  -0.075 0.940080
## pros_wc                       0.5397319  0.1575197   3.426 0.000612 ***
## cons_wc                      -0.6320353  0.2896538  -2.182 0.029107 *
## advice_wc                     0.0679477  0.1846683   0.368 0.712915
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##     Null deviance: 989.69  on 800  degrees of freedom
## Residual deviance: 369.67  on 785  degrees of freedom
## AIC: 401.67
##
## Number of Fisher Scoring iterations: 7
```

Best result yet. The model achieved an accuracy of approximately 0.864 and an AUC of 0.911, indicating strong predictive performance.

In contrast to the tenure models, recommendation behavior is well explained by workplace ratings, particularly culture, leadership, and work-life balance.

Reason for including AUC: AUC is particularly useful because it incorporates the full range of predicted probabilities and is less sensitive to threshold choice, making it a more robust measure of overall model quality.

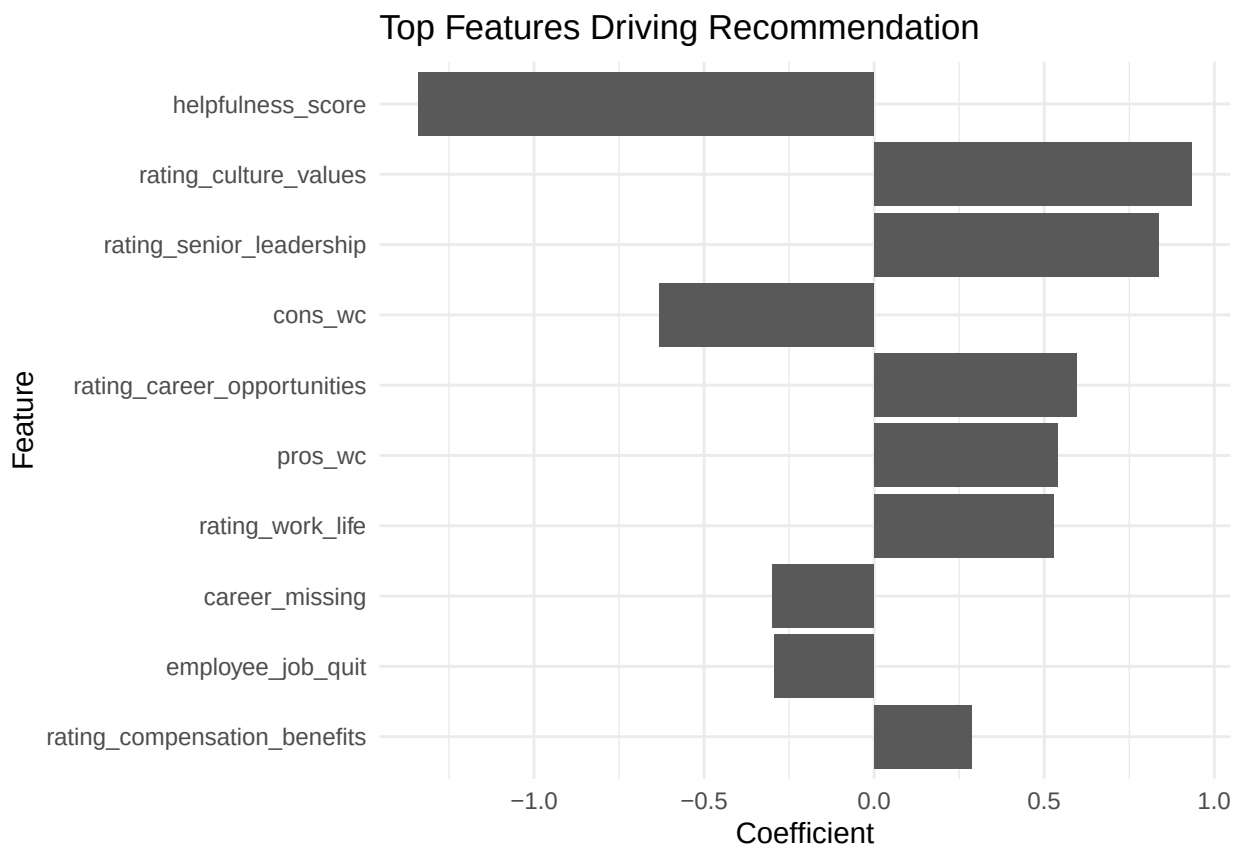
Ranking Features from the recommend logit model

```r
get_ranked_coefs <- function(model) {

  coefs <- coef(model)

  df <- data.frame(
    feature = names(coefs),
    value = as.numeric(coefs)
  )

  df <- df[df$feature != "(Intercept)", ]

  df <- df %>%
    mutate(abs_value = abs(value)) %>%
    arrange(desc(abs_value))

  df
```

```r
}

coef_df <- get_ranked_coefs(recommend_results$model)
head(coef_df, 10)
```

```
##                          feature      value abs_value
## 1             helpfulness_score -1.3405740 1.3405740
## 2         rating_culture_values  0.9335599 0.9335599
## 3      rating_senior_leadership  0.8361815 0.8361815
## 4                        cons_wc -0.6320354 0.6320354
## 5   rating_career_opportunities  0.5961229 0.5961229
## 6                        pros_wc  0.5397319 0.5397319
## 7              rating_work_life  0.5280146 0.5280146
## 8                career_missing -0.2990364 0.2990364
## 9             employee_job_quit -0.2932859 0.2932859
## 10 rating_compensation_benefits  0.2858576 0.2858576
```

```r
ggplot(coef_df[1:10, ], aes(x = reorder(feature, abs_value), y = value)) +
  geom_col() +
  coord_flip() +
  theme_minimal() +
  labs(
    title = "Top Features Driving Recommendation",
    x = "Feature",
    y = "Coefficient"
  )
```



Rating variables rank the highest. Interestingly, helpfulness score is strongly negatively correlated with recommendation - more helpful reviews tend to be more critical (not likely to be recommended).

Sidebar: Rating correlation matrix

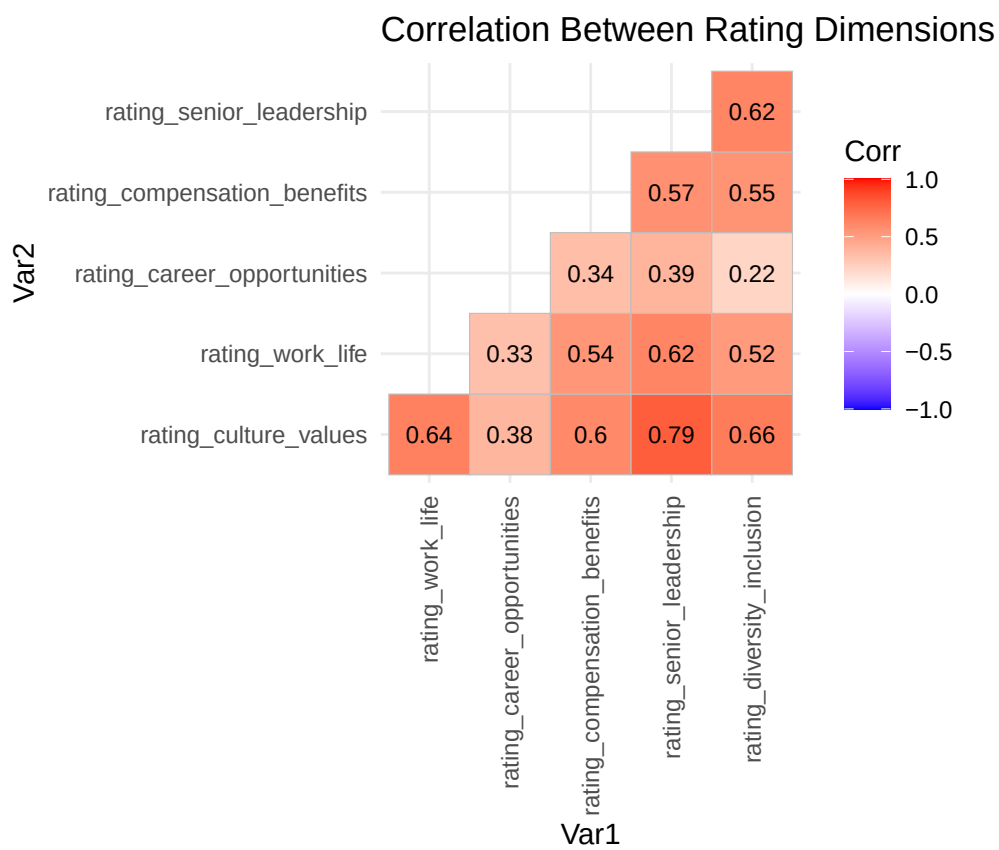
```
rating_features <- c(
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion"
)

cor_mat <- cor(data[rating_features], use = "pairwise.complete.obs")
library(ggcorrplot)
```

```
## Warning: package 'ggcorrplot' was built under R version 4.5.3
```

```
cor_mat <- cor(data[rating_features], use = "pairwise.complete.obs")

ggcorrplot(
  cor_mat,
  type = "lower",
```

```
  lab = TRUE,
  lab_size = 3,
  colors = c("blue", "white", "red"),
  title = "Correlation Between Rating Dimensions"
) +
  theme_minimal() + theme(axis.text.x = element_text(angle = 90, vjust = 0.5, hjust = 1))
```

```
## Warning: `aes_string()` was deprecated in ggplot2 3.0.0.
## i Please use tidy evaluation idioms with `aes()`.
## i See also `vignette("ggplot2-in-packages")` for more information.
## i The deprecated feature was likely used in the ggcorrplot package.
##   Please report the issue at <https://github.com/kassambara/ggcorrplot/issues>.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

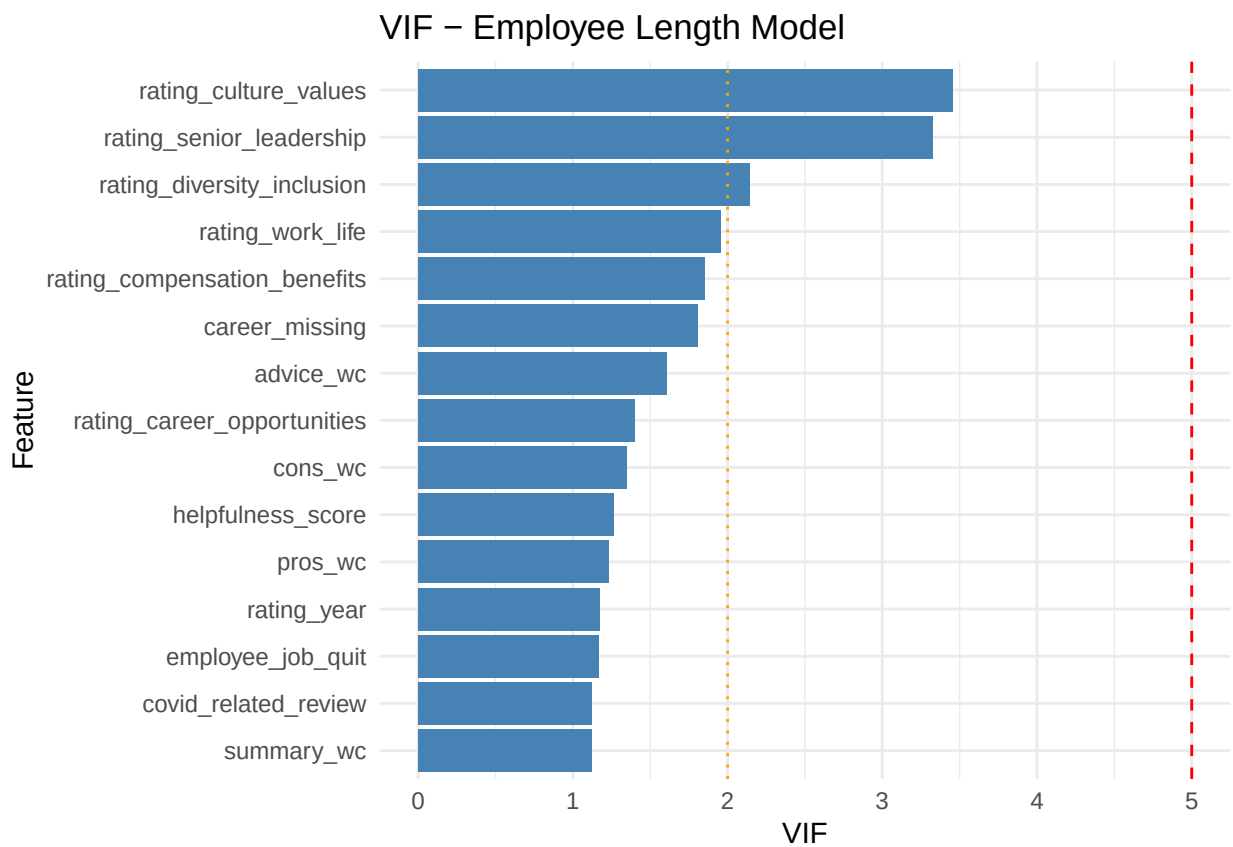


Correlation analysis shows that workplace rating variables are moderately correlated, indicating the presence of multicollinearity. As a result, coefficient estimates should be interpreted as partial effects rather than independent contributions of each feature.

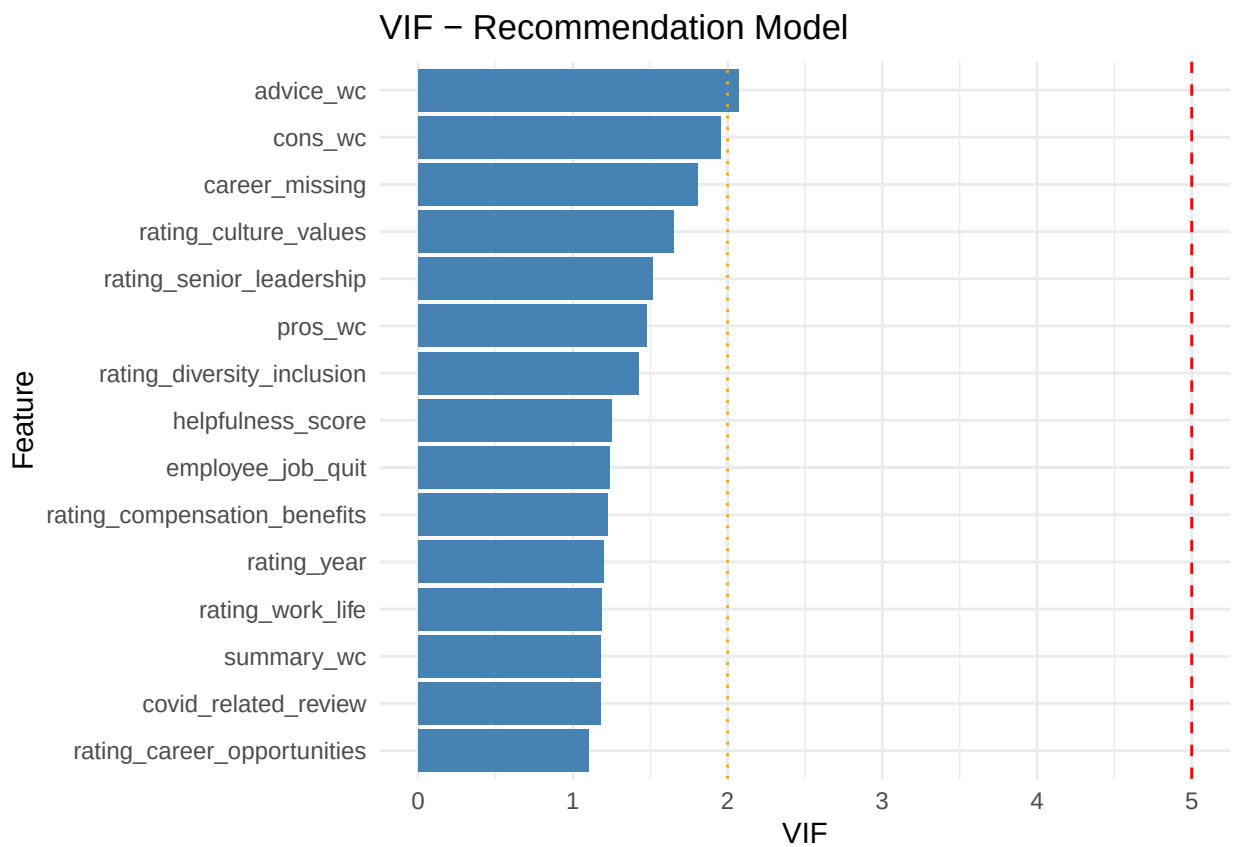
```
plot_vif <- function(model, title = "VIF Plot") {
  library(car)
  library(dplyr)
  library(ggplot2)
```

```r
  vif_vals <- vif(model)

  vif_df <- data.frame(
    feature = names(vif_vals),
    vif = as.numeric(vif_vals)
  ) %>%
    arrange(desc(vif))

  ggplot(vif_df, aes(x = reorder(feature, vif), y = vif)) +
    geom_col(fill = "steelblue") +
    coord_flip() +
    geom_hline(yintercept = 5, linetype = "dashed", color = "red") +
    geom_hline(yintercept = 2, linetype = "dotted", color = "orange") +
    theme_minimal() +
    labs(
      title = title,
      x = "Feature",
      y = "VIF"
    )
}

plot_vif(tenure_lm_results$model, "VIF - Employee Length Model")
```

```
## Loading required package: carData

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode

## The following object is masked from 'package:purrr':
##
##     some
```




```r
plot_vif(recommend_results$model, "VIF - Recommendation Model")
```



The employee_length model shows moderate multicollinearity among workplace rating variables, with VIF values around 3–3.5 for culture and senior leadership. This reflects the inherent correlation between different aspects of workplace evaluation. However, all VIF values remain below common thresholds of concern (VIF > 5), indicating that multicollinearity is present but not severe.

The recommendation model exhibits low VIF values (all below 2.5), indicating minimal multicollinearity. This suggests that the estimated effects of workplace features are stable and can be interpreted with confidence.

Regularization attempts with LASSO, Ridge and Elastic Net on recommendation model (even though multicollinearity is not that severe)

```
run_glmnet_logit <- function(df, feature_cols, target = "flags_recommend_frend", alpha = 1, seed = 123)
  df_model <- df %>%
    dplyr::select(dplyr::all_of(c(target, feature_cols))) %>%
    dplyr::filter(!is.na(.data[[target]]))

  df_model[[target]] <- factor(df_model[[target]], levels = c("NEGATIVE", "POSITIVE"))

  split_obj <- split_data(df_model, target, seed = seed)
  train_df <- split_obj$train
  test_df  <- split_obj$test
```

```r
scaled <- scale_train_test(train_df, test_df, feature_cols)
train_scaled <- scaled$train
test_scaled  <- scaled$test

train_scaled[[target]] <- factor(train_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))
test_scaled[[target]]  <- factor(test_scaled[[target]], levels = c("NEGATIVE", "POSITIVE"))

x_train <- model.matrix(
  as.formula(paste(target, "~", paste(feature_cols, collapse = " + "))),
  data = train_scaled
)[ , -1, drop = FALSE]

x_test <- model.matrix(
  as.formula(paste(target, "~", paste(feature_cols, collapse = " + "))),
  data = test_scaled
)[ , -1, drop = FALSE]

y_train <- ifelse(train_scaled[[target]] == "POSITIVE", 1, 0)
y_test  <- ifelse(test_scaled[[target]] == "POSITIVE", 1, 0)

set.seed(seed)
cv_fit <- glmnet::cv.glmnet(
  x_train, y_train,
  family = "binomial",
  alpha = alpha
)

prob_preds <- as.numeric(
  predict(cv_fit, s = "lambda.min", newx = x_test, type = "response")
)

class_preds <- ifelse(prob_preds >= 0.5, 1, 0)
acc <- mean(class_preds == y_test, na.rm = TRUE)

roc_obj <- pROC::roc(response = y_test, predictor = prob_preds)
auc_val <- as.numeric(pROC::auc(roc_obj))

coef_tbl <- as.matrix(coef(cv_fit, s = "lambda.min")) %>%
  as.data.frame()
colnames(coef_tbl) <- "estimate"
coef_tbl$term <- rownames(coef_tbl)

coef_tbl <- coef_tbl %>%
  dplyr::filter(term != "(Intercept)", estimate != 0) %>%
  dplyr::arrange(desc(abs(estimate)))

list(
  cv_model = cv_fit,
  lambda_min = cv_fit$lambda.min,
  lambda_1se = cv_fit$lambda.1se,
  auc = auc_val,
  accuracy = acc,
  coefficients = coef_tbl,
```

```r
    probabilities = prob_preds,
    actual = y_test
  )
}

lasso_rec <- run_glmnet_logit(data, features_base, alpha = 1)
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```r
ridge_rec <- run_glmnet_logit(data, features_base, alpha = 0)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
enet_rec  <- run_glmnet_logit(data, features_base, alpha = 0.5)
```

```
## Setting levels: control = 0, case = 1
## Setting direction: controls < cases
```

```r
reg_compare <- data.frame(
  model = c("Logistic", "LASSO", "Ridge", "Elastic Net"),
  auc = c(
    recommend_results$auc,
    lasso_rec$auc,
    ridge_rec$auc,
    enet_rec$auc
  ),
  accuracy = c(
    recommend_results$accuracy,
    lasso_rec$accuracy,
    ridge_rec$accuracy,
    enet_rec$accuracy
  )
)

reg_compare
```

```
##         model       auc  accuracy
## 1    Logistic 0.9117368 0.8643216
## 2       LASSO 0.9199335 0.8643216
## 3       Ridge 0.9114992 0.8643216
## 4 Elastic Net 0.9186268 0.8643216
```

Regularization barely changed performance. Saving LASSO for sentiment add-on later on.

```r
#saveRDS(lasso_rec, "models/best_recommend_base_model.rds")
#saveRDS(tenure_ord_results, "models/employee_length_bin_model.rds")
```

Does adding sentiment features help?

```
data$sent_vader_avg <- rowMeans(
  data[, c(
    "vader_summary_compound",
    "vader_pros_compound",
    "vader_cons_compound",
    "vader_advice_compound"
  )],
  na.rm = TRUE
)

data$sent_vader_sd <- apply(
  data[, c("vader_summary_compound",
    "vader_pros_compound",
    "vader_cons_compound",
    "vader_advice_compound")],
  1,
  sd,
  na.rm = TRUE
)

data$sent_distilbert_avg <- rowMeans(
  data[, c(
    "distilbert_summary_signed",
    "distilbert_pros_signed",
    "distilbert_cons_signed",
    "distilbert_advice_signed"
  )],
  na.rm = TRUE
)

data$sent_distilbert_sd <- apply(
  data[, c("distilbert_summary_signed",
         "distilbert_pros_signed",
         "distilbert_cons_signed",
         "distilbert_advice_signed")],
  1,
  sd,
  na.rm = TRUE
)

data$sent_roberta_avg <- rowMeans(
  data[, c(
    "roberta_summary_signed",
    "roberta_pros_signed",
    "roberta_cons_signed",
    "roberta_advice_signed"
  )],
  na.rm = TRUE
)

data$sent_roberta_sd <- apply(
```

```r
  data[, c(    "roberta_summary_signed",
    "roberta_pros_signed",
    "roberta_cons_signed",
    "roberta_advice_signed")],
  1,
  sd,
  na.rm = TRUE
)

features_vader <- c(
  features_base,
  "sent_vader_avg",
  "sent_vader_sd"
)

features_distilbert <- c(
  features_base,
  "sent_distilbert_avg",
  "sent_distilbert_sd"
)

features_roberta <- c(
  features_base,
  "sent_roberta_avg",
  "sent_roberta_sd"
)

feature_sets_avg_only <- list(
  structured_only = features_base,
  vader = c(features_base, "sent_vader_avg"),
  distilbert = c(features_base, "sent_distilbert_avg"),
  roberta = c(features_base, "sent_roberta_avg")
)


base_only <- features_base

feature_sets <- list(
  baseline = base_only,
  vader = features_vader,
  distilbert = features_distilbert,
  roberta = features_roberta
)
```

Employee Length model Comparison Baseline vs Sentiment-add-on

```r
tenure_ord_compare <- lapply(names(feature_sets), function(name) {
  feats <- feature_sets[[name]]

  res <- run_tenure_ordinal(
    df = data,
```

```r
    feature_cols = feats,
    target = "employee_length_bin"
  )

  data.frame(
    model = name,
    accuracy = res$accuracy
  )
})

tenure_ord_compare_df <- do.call(rbind, tenure_ord_compare)
tenure_ord_compare_df
```

```
##        model  accuracy
## 1   baseline 0.4070352
## 2      vader 0.4271357
## 3 distilbert 0.4321608
## 4    roberta 0.3819095
```

While sentiment features slightly improve tenure prediction accuracy, the overall performance remains low, indicating that employee tenure is only weakly associated with both structured workplace ratings and textual sentiment.

RoBERTa underperformed relative to both the baseline and other sentiment models. While truncation during inference may contribute to information loss, the more likely cause is domain mismatch, as the model was trained on short-form social media text rather than longer, structured employee reviews.

Recommendation Model Comparison: Baseline vs Sentiment-add-on

```r
recommend_glmnet_compare <- lapply(names(feature_sets), function(name) {

  feats <- feature_sets[[name]]

  res <- run_glmnet_logit(
    df = data,
    feature_cols = feats,
    target = "flags_recommend_frend",
    alpha = 1
  )

  data.frame(
    model = name,
    auc = res$auc,
    accuracy = res$accuracy
  )
})
```

```
## Setting levels: control = 0, case = 1
```

```
## Setting direction: controls < cases
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases

## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```
recommend_glmnet_compare_df <- do.call(rbind, recommend_glmnet_compare)
recommend_glmnet_compare_df
```

```
##        model       auc  accuracy
## 1   baseline 0.9199335 0.8643216
## 2      vader 0.9299121 0.8844221
## 3 distilbert 0.9164885 0.8743719
## 4    roberta 0.9278926 0.8643216
```

Recommendation outcomes are highly predictable, with AUC values exceeding 0.91 across all models. This indicates that structured workplace features, such as ratings, already capture most of the signal underlying recommendation decisions.

While the inclusion of sentiment features led to modest improvements—most notably with VADER, which achieved the highest AUC—these gains were relatively small. This suggests that textual sentiment provides limited additional information beyond structured ratings.

Overall, the results indicate that employee recommendation decisions are largely aligned with structured evaluations, and that sentiment extracted from review text offers only marginal incremental predictive value.

Best Features from LASSO VADER model

```
best_recommend_lasso <- run_glmnet_logit(
  df = data,
  feature_cols = feature_sets[["vader"]],
  target = "flags_recommend_frend",
  alpha = 1
)
```

```
## Setting levels: control = 0, case = 1

## Setting direction: controls < cases
```

```
best_recommend_lasso$coefficients
```

```
##                                 estimate                        term
## helpfulness_score            -0.99533614            helpfulness_score
## sent_vader_avg                0.86081530               sent_vader_avg
## rating_senior_leadership      0.76768482     rating_senior_leadership
## rating_culture_values         0.75817834        rating_culture_values
## rating_career_opportunities   0.54384001  rating_career_opportunities
## cons_wc                      -0.53996997                      cons_wc
## rating_work_life              0.46228976             rating_work_life
## pros_wc                       0.41182121                      pros_wc
## career_missing               -0.28009534               career_missing
## employee_job_quit            -0.17219313            employee_job_quit
## rating_compensation_benefits  0.11830792 rating_compensation_benefits
## summary_wc                   -0.06508652                   summary_wc
## rating_year                   0.01922134                  rating_year
```

LASSO regularization revealed that structured rating variables—particularly senior leadership, culture, and career opportunities—are the primary drivers of recommendation outcomes.

Interestingly, the average VADER sentiment score remained one of the strongest predictors, indicating that textual sentiment contributes additional predictive signal beyond structured ratings.

Text-based features such as the word counts of pros and cons sections also played a meaningful role, with longer positive sections increasing the likelihood of recommendation and longer negative sections decreasing it.

Overall, while structured ratings dominate prediction, sentiment and textual characteristics provide complementary information.

Cluster Chaos

Clustering Prep for review-level clustering, company-level would be (was) too weak

For selecting features to cluster on, avoid selecting response variables and selecting all predictors is generally not a good idea (can degrade clustering performance due to curse of dimensionality).

Select only vader_avg for sentiment because it was among the top performing features in the previous supervised models.

```r
cluster_structured <- c(
  "rating_culture_values",
  "rating_work_life",
  "rating_career_opportunities",
  "rating_compensation_benefits",
  "rating_senior_leadership",
  "rating_diversity_inclusion"
)

cluster_features <- c(
  cluster_structured,
  "sent_vader_avg",
  "pros_wc",
  "cons_wc",
  "helpfulness_score"
)
```

Scaling clusters

```
cluster_df <- data %>%
  dplyr::select(all_of(cluster_features)) %>%
  na.omit()

cluster_scaled <- scale(cluster_df)
apply(cluster_scaled, 2, mean)
```

```
##        rating_culture_values             rating_work_life
##                 3.204219e-17                -2.312170e-17
##  rating_career_opportunities rating_compensation_benefits
##                 1.791681e-16                -1.255168e-16
##     rating_senior_leadership   rating_diversity_inclusion
##                 1.052276e-16                -8.186615e-17
##               sent_vader_avg                      pros_wc
##                 1.405126e-19                 4.039087e-17
##                      cons_wc            helpfulness_score
##                -5.772466e-17                 5.127040e-17
```

```
apply(cluster_scaled, 2, sd)
```

```
##        rating_culture_values             rating_work_life
##                            1                            1
##  rating_career_opportunities rating_compensation_benefits
##                            1                            1
##     rating_senior_leadership   rating_diversity_inclusion
##                            1                            1
##               sent_vader_avg                      pros_wc
##                            1                            1
##                      cons_wc            helpfulness_score
##                            1                            1
```

After selecting the clustering variables, a review-level clustering dataset is created. All selected variables are standardized prior to clustering so that each feature can be comparably used in Euclidean distance calculations. This step is important because the variables operate on different scales, including ratings, word counts, helpfulness, and sentiment.

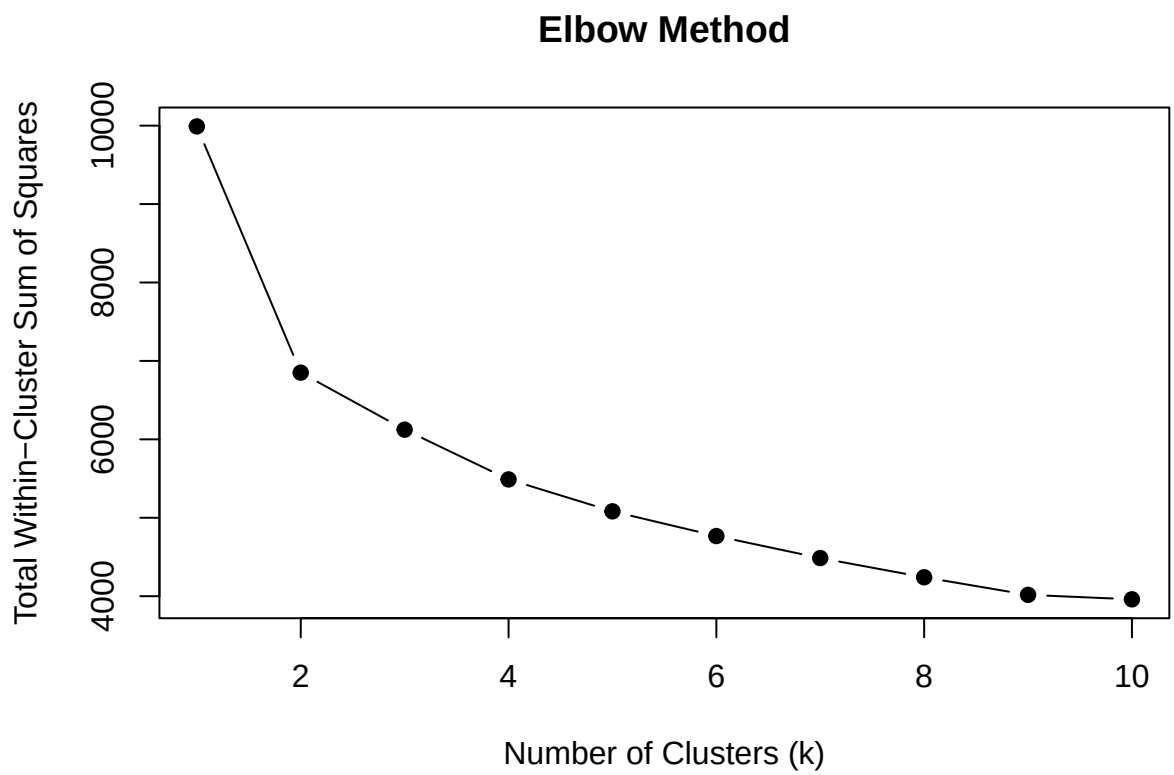
Standardization went well looking at cluster_scaled

Choosing K (number of clusters)

Elbow Method

```
set.seed(123)

wss <- sapply(1:10, function(k) {
  kmeans(cluster_scaled, centers = k, nstart = 25)$tot.withinss
})

plot(1:10, wss,
```

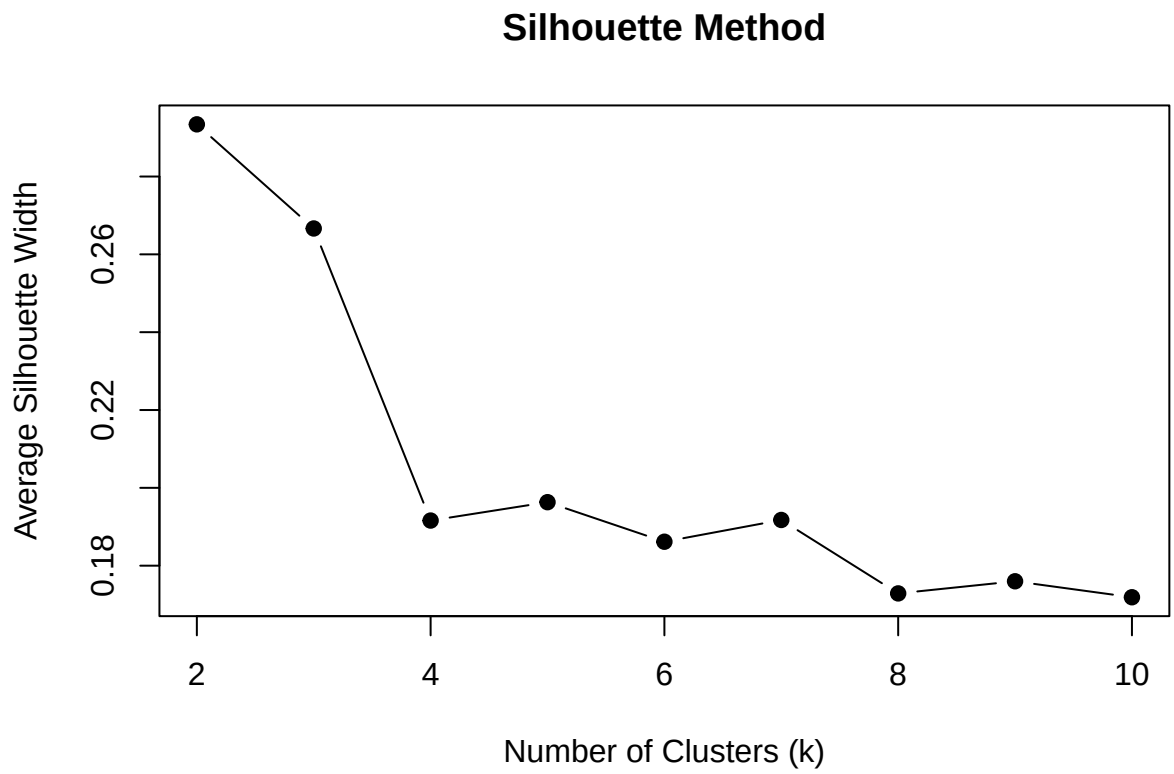
```
     type = "b",
     pch = 19,
     xlab = "Number of Clusters (k)",
     ylab = "Total Within-Cluster Sum of Squares",
     main = "Elbow Method")
```



Candidate elbow points are 2, 3, 4. Let's try silhouette method to lock-in the k

Silhouette Method

```
library(cluster)

sil_scores <- sapply(2:10, function(k) {
  km <- kmeans(cluster_scaled, centers = k, nstart = 25)
  ss <- silhouette(km$cluster, dist(cluster_scaled))
  mean(ss[, 3])
})

plot(2:10, sil_scores,
     type = "b",
     pch = 19,
     xlab = "Number of Clusters (k)",
     ylab = "Average Silhouette Width",
     main = "Silhouette Method")
```



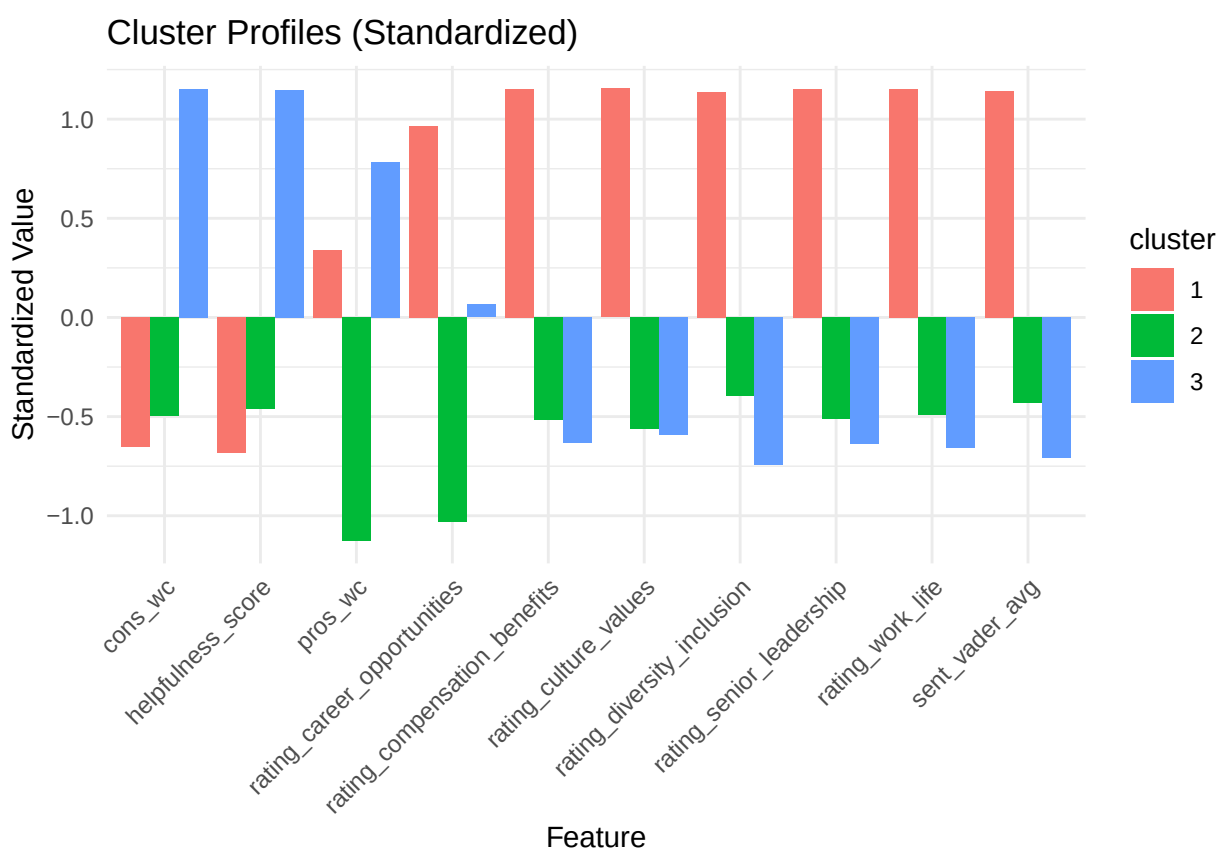
The silhouette method indicated that k = 2 provides the strongest cluster separation. However, k = 3 was selected to capture more nuanced structure in the data, as it still maintains reasonable separation while allowing differentiation between positive, mixed, and negative workplace experiences. This aligns better with the goals of the analysis.

K-Means attempt

```
set.seed(123)

kmeans_model <- kmeans(cluster_scaled, centers = 3, nstart = 25)

cluster_df$cluster <- factor(kmeans_model$cluster)

set.seed(123)

kmeans_model <- kmeans(cluster_scaled, centers = 3, nstart = 25)

cluster_df$cluster <- factor(kmeans_model$cluster)

cluster_summary <- cluster_df %>%
  dplyr::group_by(cluster) %>%
  dplyr::summarise(across(everything(), mean))

cluster_summary
```

```
## # A tibble: 3 x 11
##   cluster rating_culture_values rating_work_life rating_career_opportunities
##   <fct>                   <dbl>            <dbl>                       <dbl>
## 1 1                        3.97             3.83                        3.59
## 2 2                        1.51             1.88                        2.74
## 3 3                        1.47             1.68                        3.20
## # i 7 more variables: rating_compensation_benefits <dbl>,
## #   rating_senior_leadership <dbl>, rating_diversity_inclusion <dbl>,
## #   sent_vader_avg <dbl>, pros_wc <dbl>, cons_wc <dbl>, helpfulness_score <dbl>
```

```r
cluster_scaled_summary <- cluster_summary %>%
  mutate(across(-cluster, scale))

cluster_long <- cluster_scaled_summary %>%
  pivot_longer(-cluster, names_to = "feature", values_to = "value")

ggplot(cluster_long, aes(x = feature, y = value, fill = cluster)) +
  geom_col(position = "dodge") +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 45, hjust = 1)) +
  labs(
    title = "Cluster Profiles (Standardized)",
    x = "Feature",
    y = "Standardized Value"
  )
```



Cluster 1 (High Satisfaction): Above-average ratings across all dimensions, positive sentiment, and more

pros than cons. Represents clearly satisfied employees.

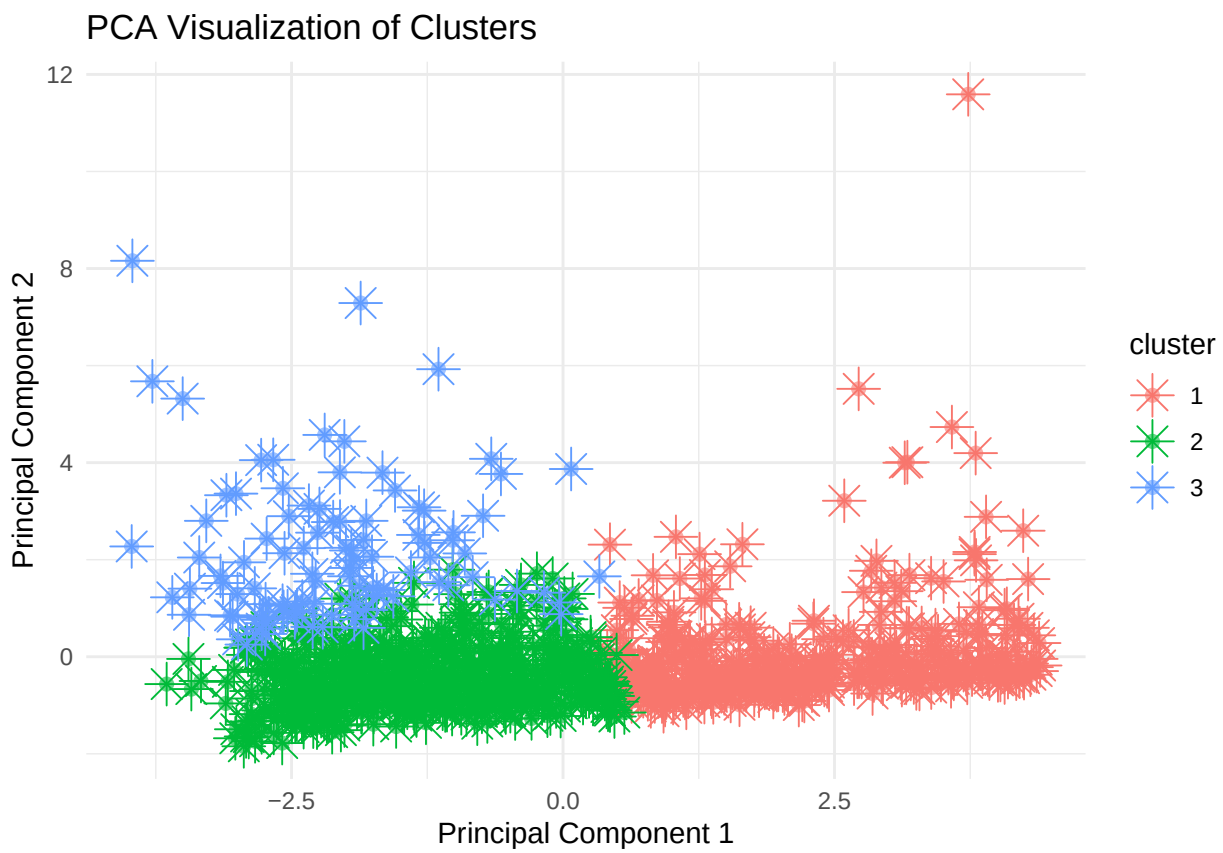
Cluster 2 (Disengaged Negative): Below-average ratings and sentiment, with low pros and moderate cons. Reflects generally dissatisfied but less engaged reviewers.

Cluster 3 (Engaged Critics): Below-average ratings and sentiment, but high pros, cons, and helpfulness. Indicates dissatisfied yet highly engaged reviewers providing detailed feedback.

```r
pca_res <- prcomp(cluster_scaled, center = FALSE, scale. = FALSE)
pca_df <- as.data.frame(pca_res$x)

pca_df$cluster <- cluster_df$cluster

summary(pca_res)
```

```
## Importance of components:
##                           PC1    PC2     PC3     PC4     PC5    PC6     PC7
## Standard deviation     2.0648 1.1883 0.98687 0.83011 0.79449 0.7443 0.68721
## Proportion of Variance 0.4263 0.1412 0.09739 0.06891 0.06312 0.0554 0.04723
## Cumulative Proportion  0.4263 0.5675 0.66493 0.73384 0.79696 0.8524 0.89959
##                            PC8     PC9    PC10
## Standard deviation     0.67416 0.59221 0.44593
## Proportion of Variance 0.04545 0.03507 0.01989
## Cumulative Proportion  0.94504 0.98011 1.00000
```

```r
ggplot(pca_df, aes(x = PC1, y = PC2, color = cluster)) +
  geom_point(alpha = 0.6, size = 2) +
  labs(
    title = "PCA Visualization of Clusters",
    x = "Principal Component 1",
    y = "Principal Component 2"
  ) +
  theme_minimal() + stat_summary(
    fun = mean,
    geom = "point",
    size = 5,
    shape = 8,
    aes(group = cluster)
)
```



Principal Component Analysis revealed that the first component explains approximately 42.6% of the variance and primarily captures overall workplace quality, separating high-rated environments from low-rated ones. The second component explains an additional 14.1% of the variance and differentiates between types of negative experiences.

Specifically, it separates uniformly poor workplace environments from those that offer relatively stronger career opportunities despite low ratings in culture and leadership. The PCA visualization supports the clustering results by showing clear separation between high-satisfaction reviews and more nuanced overlap among lower-rated groups.